UGANDA MARTYRS UNIVERSITY
Faculty of Science — Department of Computer Science & Information Systems

UMU EVENTS MANAGEMENT SYSTEM

Project Report

Web-Based System Programming — CSC 2202

End-of-Semester Final Assessment | Semester 2, 2025/2026
BSc IT / BCS / BScEd — Second Year

Submitted by: Group
Academic Year: 2026

GROUP MEMBERS

ID	REGISTRATION NUMBER
1	2024-B072-31778
2	2024-B072-31093
3	2024-B072-31781
4	2024-B201-31811
5	2025-B072-33409
6	2024-B072-31783

Overview

The UMU Events Management System is a full-stack, web-based application developed for Uganda Martyrs University. It provides a centralized digital platform through which the university community can create, discover, and register for events. The system eliminates the traditional challenges of scattered event informat5ion, manual RSVP tracking, and duplicate registrations by offering an intuitive, role-aware interface backed by relational database

Problem statement

Prior to this system, university event management suffered from several compounding inefficiencies:

I. Event announcements were distributed across multiple disconnected channels, causing low visibility
II. RSVP tracking was performed manually, leading to errors, duplicate and inaccurate attendance records.
III. Administrators had no centralized tool for creating, editing, or removing events.
IV. Students lacked a single place to discover upcoming events and manage their own RSVP history.
V. There were no real-time updates or alerts when details changed.

Proposed solutions

The UMU Events Management system addresses these problems by delivering:

I. A centralized web portal accessible to both administrators and students through a single URL.
II. Built-in RSVP functionality with duplicate prevention – a student cannot RSVP to the same event more than once.
III. An admin dashboard that enables full Create, Read, Update, and Delete (CRUD) control over the events and categories.
IV. Secure role-based access control distinguishing Admin and Student privileges.
V. Search and filtering capabilities allowing students to locate events by title, date, or category.

Technology Stack

Layer	**Technology**	**Purpose**
Frontend	HTML5, CSS3, VanillaJS, Google Fonts	Semantic markup, responsive layouts, DOM manipulation, DM Sans typeface
Backend	PHP 8.0+, PDO, Sessions	Server-side routing, prepared statements for SQL-injection protection, bcryt password hashing
Database	MySQL 5.7+ / MariaDB 10.4+	InnoDB engine with foreign key constraints; 3 core relational tables; seed data included
Server	WAMP (Apache 2.4, PHP 8.0.26)	Mod_rewrite for URL control, .htaccess security headers, dynamic Base_URL detaction

Three-Tier Architecture

The application follows a classic three-tier architecture separarting presentation, business logic, and data concerns

Tier	Components
Presentation	Login.php, dashboard.php, events.php, rsvp.php, index.php, register.php, user.phpp, logout.php, forget_password.php
Business logic	/app.php (BASE_URL detection), config/database.php (PDO singleton), includes/header.php (auth + navigation), includes/footer.php (scripts), Session management + role-based guards
Data	MySQL database: umu_event_management — 3 core tables (users, events, rsvp) with full referential integrity via InnoDB foreign-key constraints

Functional Modules

The system delivers nine functional modules that together satisfy all ten assessment requirements:

Module	**Description**
Dashboard	KPI overview — upcoming events count, total RSVPs, recent activity feed.
User Authentication	Student registration, login, logout, and session management with bcrypt hashing.
Event Browsing	Students view all available and upcoming events in a paginated, structured list.
RSVP Management	One-click RSVP with duplicate prevention; system tracks attendance per event.
RSVP History	Students review all events they have previously RSVP'd to.
Categories Management	Admin manages event categories (Academic, Social, Sports, Cultural, etc.).
Search & Filtering	Users search events by title, filter by date or category for quick discovery.
Alerts	Auto-generated system alerts with severity levels; mark-as-read and delete actions.
Event Management	Admin-only CRUD interface: create, edit, update, and delete events with form validation.

Schema Overview

The database umu_event_management comprises three core relational tables backed by the InnoDB engine, ensuring foreign-key constraints and full referential integrity. Seed data is included to populate all modules with realistic demo records upon installation.

Table Definitions

Table: users

Column	Type	Key	Description
id	INT AUTO_INCREMENT	PK	Unique user identifier
name	VARCHAR(100)	—	Full name of the user
email	VARCHAR(150)	UNIQUE	Login email; must be unique across all users
password	VARCHAR(255)	—	bcrypt-hashed password via password_hash()
role	ENUM	—	'admin' or 'student' — controls access guards
created_at	TIMESTAMP	—	Account creation timestamp

Table: events

Column	Type	Key	Description
id	INT AUTO_INCREMENT	PK	Unique event identifier
title	VARCHAR(200)	—	Name/title of the event
description	TEXT	—	Full event description
category	VARCHAR(100)	—	Category label (Academic, Social, Sports, etc.)
event_date	DATETIME	—	Scheduled date and time of the event
location	VARCHAR(255)	—	Physical or virtual venue of the event
created_by	INT	FK → users	References the admin user who created the event

Table: rsvp

Column	Type	Key	Description
id	INT AUTO_INCREMENT	PK	Unique RSVP record identifier
user_id	INT	FK → users	The student who submitted the RSVP
event_id	INT	FK → events	The event being RSVP'd to
rsvp_date	TIMESTAMP	—	Date and time the RSVP was submitted

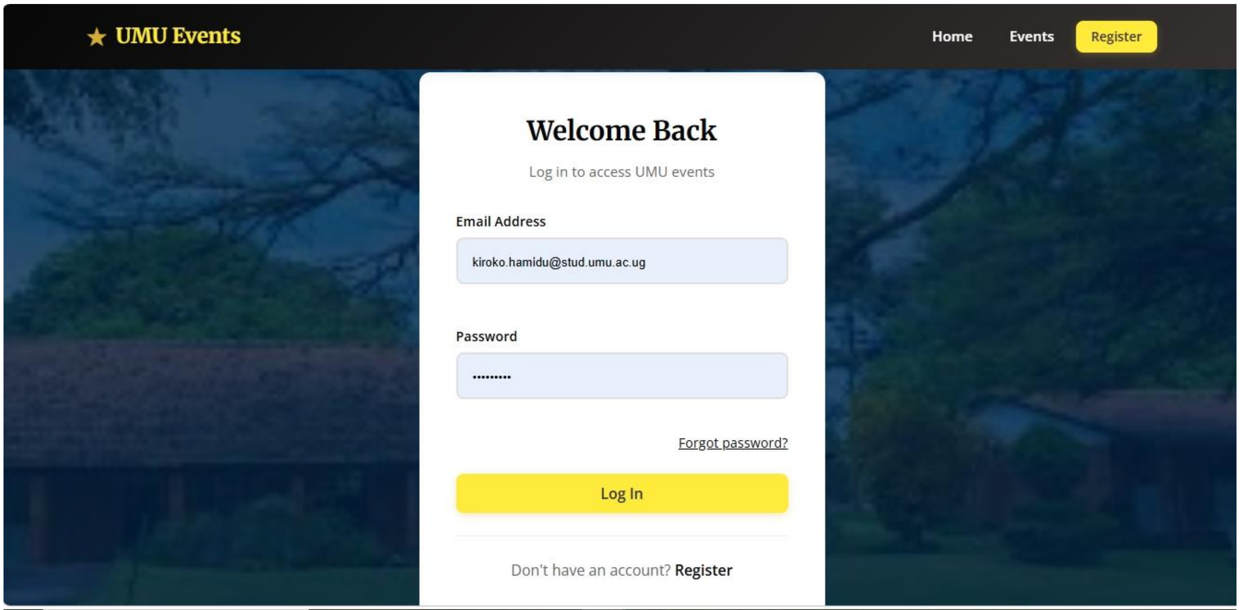
3.3 Entity Relationships

The three tables are linked through InnoDB foreign-key constraints:

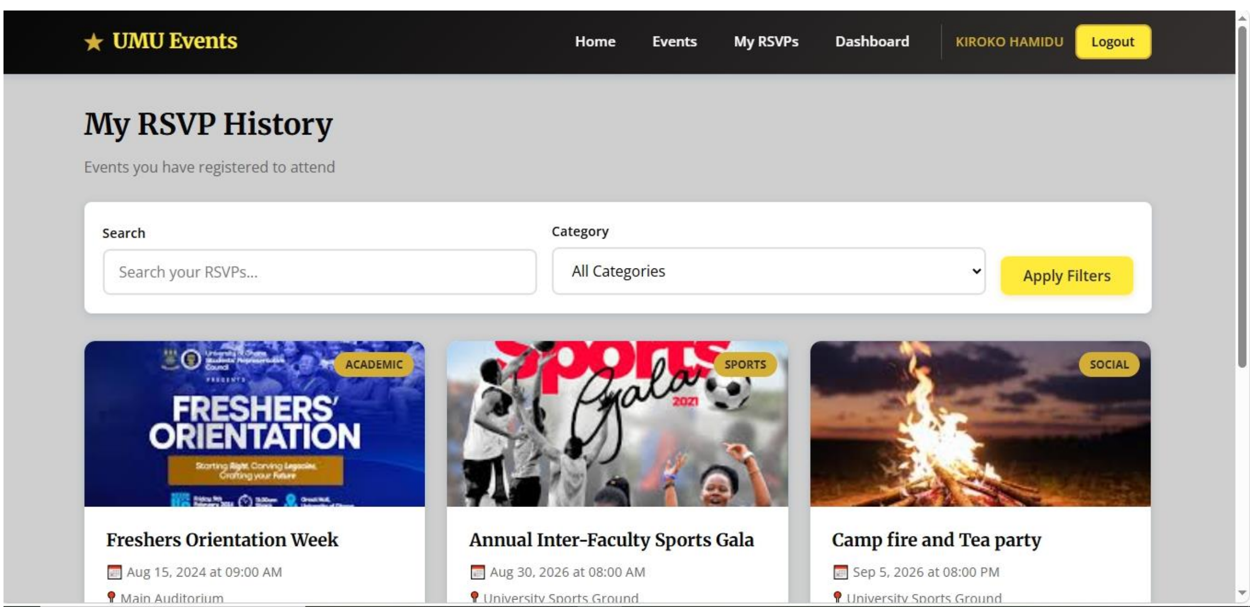
- users (1) → rsvp (many): one student can submit multiple RSVPs across different events.
- events (1) → rsvp (many): one event can receive RSVPs from many students.
- users (1) → events (many): one admin user can create multiple events.

A UNIQUE constraint on (user_id, event_id) in the rsvp table enforces the business rule that no student may RSVP to the same event more than once, at the database level rather than only in application logic

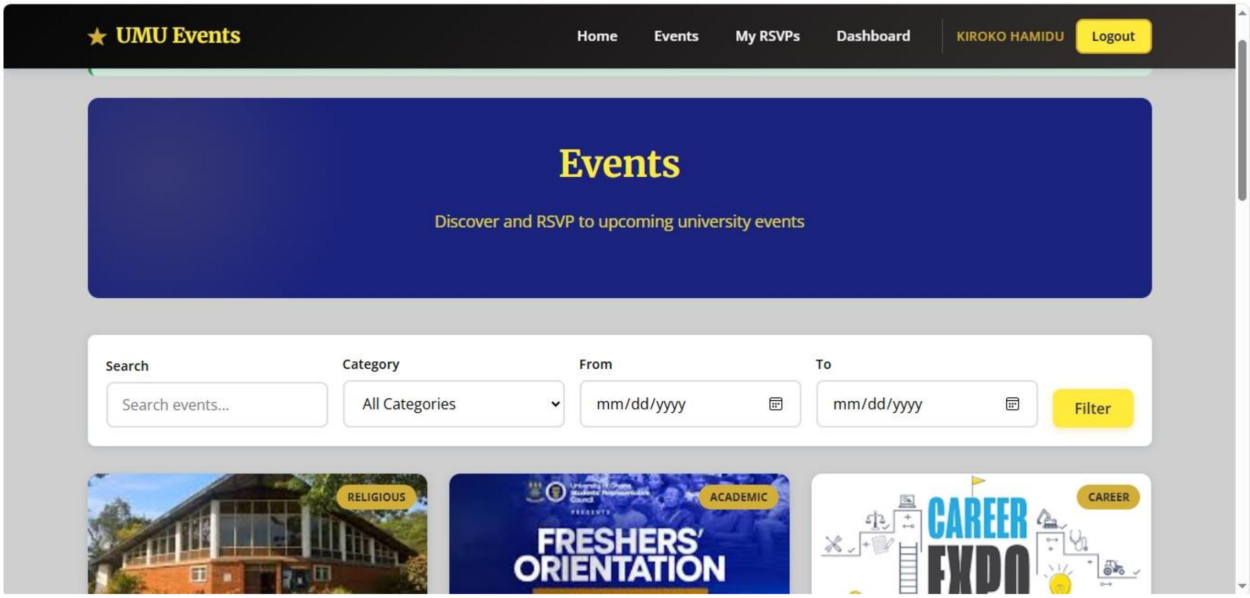
Login / Register screen



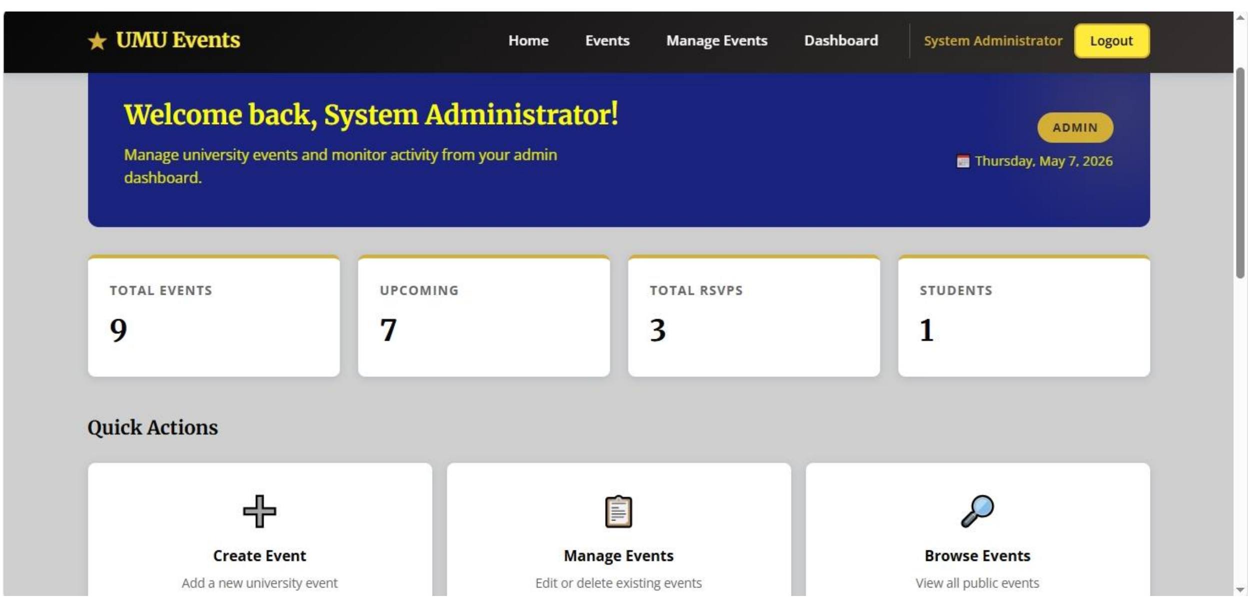
RSVP Screen



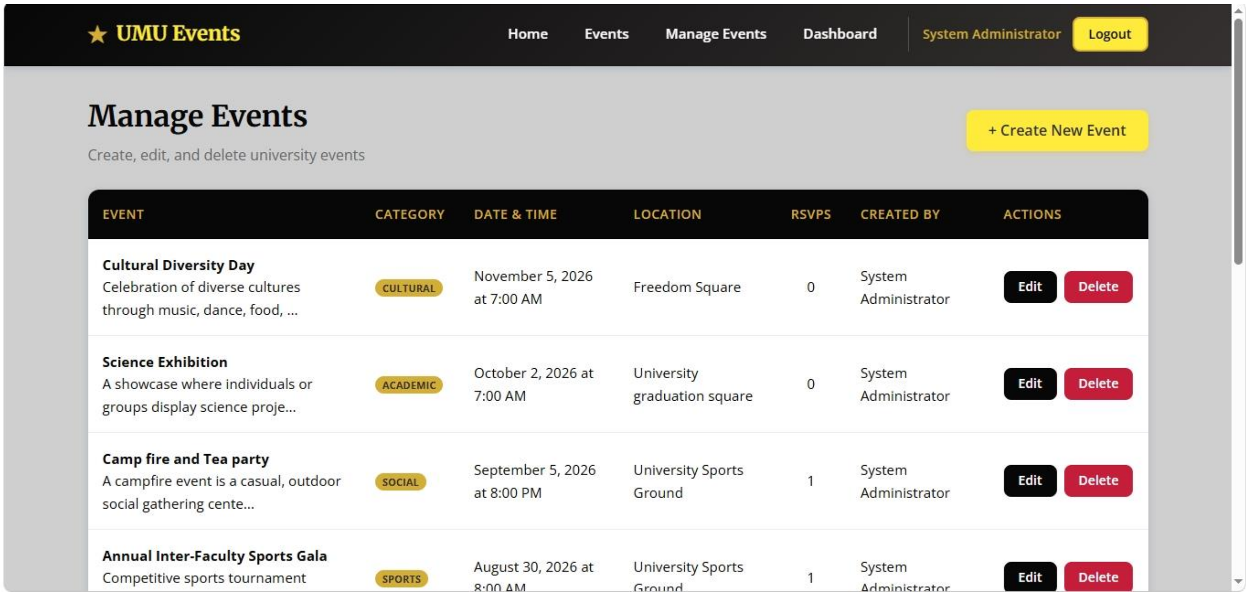
Browsing Events screen



Admin dashboard screen



Admin CRUD screen



REFLECTION ABOUT THE CHALLENGES WE FACED

The team encountered and addressed challenges during development:

I. WAMP compatibility required deeper knowledge of Apache mod_rewrite and .htaccess configuration than anticipated. Ensuring the application worked both in the project root and a named subfolder demanded systematic path abstraction.

II. Role-based session guards needed to be enforced at the server level on every protected PHP file, not only on the client, since direct URL access could bypass any JavaScript-only checks.

III. Database normalisation decisions — particularly separating event categories into their own managed entity — improved flexibility but required additional foreign-key planning to maintain referential integrity.

IV. Login password hash mismatch, this was corrected through the README credentials; documented default password as 'password' for all seeded users